Apache Spark 4.0.1 — Complete Configuration Reference

Official Apache Spark 4.0.1 documentation • 354 documented Spark configuration entries

Columns: configuration key, documented default, description, and version introduced. Conditional defaults and cluster-manager-specific behavior are retained as documented.

General

Configuration	Default	Description	Since
spark.app.name	(none)	The name of your application. This will appear in the UI and in log data.	0.9.0
spark.driver.cores	1	Number of cores to use for the driver process, only in cluster mode.	1.3.0
spark.driver.maxResultSize	1g	Limit of total size of serialized results of all partitions for each Spark action (e.g. collect) in bytes. Should be at least 1M, or 0 for unlimited. Jobs will be aborted if the total size is above this limit. Having a high limit may cause out-of-memory errors in driver (depends on spark.driver.memory and memory overhead of objects in JVM). Setting a proper limit can protect the driver from out-of-memory errors.	1.2.0
spark.driver.memory	1g	Amount of memory to use for the driver process, i.e. where SparkContext is initialized, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m , 2g). Note: In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the --driver-memory command line option or in your default properties file.	1.1.1
spark.driver.memoryOverhead	driverMemory * spark.driver.memoryOverheadFactor , with minimum of spark.driver.minMemoryOverhead	Amount of non-heap memory to be allocated per driver process in cluster mode, in MiB unless otherwise specified. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size (typically 6-10%). This option is currently supported on YARN and Kubernetes. Note: Non-heap memory includes off-heap memory (when spark.memory.offHeap.enabled=true) and memory used by other driver processes (e.g. python process that goes with a PySpark driver) and memory used by other non-driver processes running in the same container. The maximum memory size of container to running driver is determined by the sum of spark.driver.memoryOverhead and spark.driver.memory .	2.3.0
spark.driver.minMemoryOverhead	384m	The minimum amount of non-heap memory to be allocated per driver process in cluster mode, in MiB unless otherwise specified, if spark.driver.memoryOverhead is not defined. This option is currently supported on YARN and Kubernetes.	4.0.0
spark.driver.memoryOverheadFactor	0.10	Fraction of driver memory to be allocated as additional non-heap memory per driver process in cluster mode. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size. This value defaults to 0.10 except for Kubernetes non-JVM jobs, which defaults to 0.40. This is done as non-JVM tasks need more non-JVM heap space and such tasks commonly fail with "Memory Overhead Exceeded" errors. This preempts this error with a higher default. This value is ignored if spark.driver.memoryOverhead is set directly.	3.3.0
spark.driver.resource.{resourceName}.amount	0	Amount of a particular resource type to use on the driver. If this is used, you must also specify the spark.driver.resource.{resourceName}.discoveryScript for the driver to find the resource on startup.	3.0.0
spark.driver.resource.{resourceName}.discoveryScript	None	A script for the driver to run to discover a particular resource type. This should write to STDOUT a JSON string in the format of the ResourceInformation class. This has a name and an array of addresses. For a client-submitted driver, discovery script must assign different resource addresses to this driver comparing to other drivers on the same host.	3.0.0
spark.driver.resource.{resourceName}.vendor	None	Vendor of the resources to use for the driver. This option is currently only supported on Kubernetes and is actually both the vendor and domain following the Kubernetes device plugin naming convention. (e.g. For GPUs on Kubernetes this config would be set to nvidia.com or amd.com)	3.0.0
spark.resources.discoveryPlugin	org.apache.spark.resource.ResourceDiscoveryScriptPlugin	Comma-separated list of class names implementing org.apache.spark.api.resource.ResourceDiscoveryPlugin to load into the application. This is for advanced users to replace the resource discovery class with a custom implementation. Spark will try each class specified until one of them returns the resource information for that resource. It tries the discovery script last if none of the plugins return information for that resource.	3.0.0
spark.executor.memory	1g	Amount of memory to use per executor process, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m , 2g).	0.7.0
spark.executor.pyspark.memory	Not set	The amount of memory to be allocated to PySpark in each executor, in MiB unless otherwise specified. If set, PySpark memory for an executor will be limited to this amount. If not set, Spark will not limit Python's memory use and it is up to the application to avoid exceeding the overhead memory space shared with other non-JVM processes. When PySpark is run in YARN or Kubernetes, this memory is added to executor resource requests. Note: This feature is dependent on Python's resource module; therefore, the behaviors and limitations are inherited. For instance, Windows does not support resource limiting and actual resource is not limited on MacOS.	2.4.0
spark.executor.memoryOverhead	executorMemory * spark.executor.memoryOverheadFactor , with minimum of spark.executor.minMemoryOverhead	Amount of additional memory to be allocated per executor process, in MiB unless otherwise specified. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the executor size (typically 6-10%). This option is currently supported on YARN and Kubernetes. Note: Additional memory includes PySpark executor memory (when spark.executor.pyspark.memory is not configured) and memory used by other non-executor processes running in the same container. The maximum memory size of container to running executor is determined by the sum of spark.executor.memoryOverhead , spark.executor.memory , spark.memory.offHeap.size and spark.executor.pyspark.memory .	2.3.0
spark.executor.minMemoryOverhead	384m	The minimum amount of non-heap memory to be allocated per executor process, in MiB unless otherwise specified, if spark.executor.memoryOverhead is not defined. This option is currently supported on YARN and Kubernetes.	4.0.0
spark.executor.memoryOverheadFactor	0.10	Fraction of executor memory to be allocated as additional non-heap memory per executor process. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size. This value defaults to 0.10 except for Kubernetes non-JVM jobs, which defaults to 0.40. This is done as non-JVM tasks need more non-JVM heap space and such tasks commonly fail with "Memory Overhead Exceeded" errors. This preempts this error with a higher default. This value is ignored if spark.executor.memoryOverhead is set directly.	3.3.0

Configuration	Default	Description	Since
spark.executor.resource.{resourceName}.amount	0	Amount of a particular resource type to use per executor process. If this is used, you must also specify the spark.executor.resource.{resourceName}.discoveryScript for the executor to find the resource on startup.	3.0.0
spark.executor.resource.{resourceName}.discoveryScript	None	A script for the executor to run to discover a particular resource type. This should write to STDOUT a JSON string in the format of the ResourceInformation class. This has a name and an array of addresses.	3.0.0
spark.executor.resource.{resourceName}.vendor	None	Vendor of the resources to use for the executors. This option is currently only supported on Kubernetes and is actually both the vendor and domain following the Kubernetes device plugin naming convention. (e.g. For GPUs on Kubernetes this config would be set to nvidia.com or amd.com)	3.0.0
spark.extraListeners	(none)	A comma-separated list of classes that implement SparkListener ; when initializing SparkContext, instances of these classes will be created and registered with Spark's listener bus. If a class has a single-argument constructor that accepts a SparkConf, that constructor will be called; otherwise, a zero-argument constructor will be called. If no valid constructor can be found, the SparkContext creation will fail with an exception.	1.3.0
spark.local.dir	/tmp	Directory to use for "scratch" space in Spark, including map output files and RDDs that get stored on disk. This should be on a fast, local disk in your system. It can also be a comma-separated list of multiple directories on different disks. Note: This will be overridden by SPARK_LOCAL_DIRS (Standalone) or LOCAL_DIRS (YARN) environment variables set by the cluster manager.	0.5.0
spark.logConf	false	Logs the effective SparkConf as INFO when a SparkContext is started.	0.9.0
spark.master	(none)	The cluster manager to connect to. See the list of allowed master URL's .	0.9.0
spark.submit.deployMode	client	The deploy mode of Spark driver program, either "client" or "cluster", Which means to launch driver program locally ("client") or remotely ("cluster") on one of the nodes inside the cluster.	1.5.0
spark.log.callerContext	(none)	Application information that will be written into Yarn RM log/HDFS audit log when running on Yarn/HDFS. Its length depends on the Hadoop configuration hadoop.caller.context.max.size . It should be concise, and typically can have up to 50 characters.	2.2.0
spark.log.level	(none)	When set, overrides any user-defined log settings as if calling SparkContext.setLogLevel() at Spark startup. Valid log levels include: "ALL", "DEBUG", "ERROR", "FATAL", "INFO", "OFF", "TRACE", "WARN".	3.5.0
spark.driver.supervise	false	If true, restarts the driver automatically if it fails with a non-zero exit status. Only has effect in Spark standalone mode.	1.3.0
spark.driver.timeout	0min	A timeout for Spark driver in minutes. 0 means infinite. For the positive time value, terminate the driver with the exit code 124 if it runs after timeout duration. To use, it's required to set spark.plugins with org.apache.spark.deploy.DriverTimeoutPlugin .	4.0.0
spark.driver.log.localDir	(none)	Specifies a local directory to write driver logs and enable Driver Log UI Tab.	4.0.0
spark.driver.log.dfsDir	(none)	Base directory in which Spark driver logs are synced, if spark.driver.log.persistToDfs.enabled is true. Within this base directory, each application logs the driver logs to an application specific file. Users may want to set this to a unified location like an HDFS directory so driver log files can be persisted for later usage. This directory should allow any Spark user to read/write files and the Spark History Server user to delete files. Additionally, older logs from this directory are cleaned by the Spark History Server if spark.history.fs.driverlog.cleaner.enabled is true and, if they are older than max age configured by setting spark.history.fs.driverlog.cleaner.maxAge .	3.0.0
spark.driver.log.persistToDfs.enabled	false	If true, spark application running in client mode will write driver logs to a persistent storage, configured in spark.driver.log.dfsDir . If spark.driver.log.dfsDir is not configured, driver logs will not be persisted. Additionally, enable the cleaner by setting spark.history.fs.driverlog.cleaner.enabled to true in Spark History Server .	3.0.0
spark.driver.log.layout	%d{yy/MM/dd HH:mm:ss.SSS} %t %p %c{1}: %m%n%ex	The layout for the driver logs that are synced to spark.driver.log.localDir and spark.driver.log.dfsDir . If this is not configured, it uses the layout for the first appender defined in log4j2.properties. If that is also not configured, driver logs use the default layout.	3.0.0
spark.driver.log.allowErasureCoding	false	Whether to allow driver logs to use erasure coding. On HDFS, erasure coded files will not update as quickly as regular replicated files, so they make take longer to reflect changes written by the application. Note that even if this is true, Spark will still not force the file to use erasure coding, it will simply use file system defaults.	3.0.0
spark.decommission.enabled	false	When decommission enabled, Spark will try its best to shut down the executor gracefully. Spark will try to migrate all the RDD blocks (controlled by spark.storage.decommission.rddBlocks.enabled) and shuffle blocks (controlled by spark.storage.decommission.shuffleBlocks.enabled) from the decommissioning executor to a remote executor when spark.storage.decommission.enabled is enabled. With decommission enabled, Spark will also decommission an executor instead of killing when spark.dynamicAllocation.enabled enabled.	3.1.0
spark.executor.decommission.killInterval	(none)	Duration after which a decommissioned executor will be killed forcefully by an outside (e.g. non-spark) service.	3.1.0
spark.executor.decommission.forceKillTimeout	(none)	Duration after which a Spark will force a decommissioning executor to exit. This should be set to a high value in most situations as low values will prevent block migrations from having enough time to complete.	3.2.0
spark.executor.decommission.signal	PWR	The signal that used to trigger the executor to start decommission.	3.2.0
spark.executor.maxNumFailures	numExecutors * 2, with minimum of 3	The maximum number of executor failures before failing the application. This configuration only takes effect on YARN and Kubernetes.	3.5.0
spark.executor.failuresValidityInterval	(none)	Interval after which executor failures will be considered independent and not accumulate towards the attempt count. This configuration only takes effect on YARN and Kubernetes.	3.5.0
spark.driver.extraClassPath	(none)	Extra classpath entries to prepend to the classpath of the driver. Note: In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the --driver-class-path command line option or in your default properties file.	1.0.0
spark.driver.defaultJavaOptions	(none)	A string of default JVM options to prepend to spark.driver.extraJavaOptions . This is intended to be set by administrators. For instance, GC settings or other logging. Note that it is illegal to set maximum heap size (-Xmx) settings with this option. Maximum heap size settings can be set with spark.driver.memory in the cluster mode and through the --driver-memory command line option in the client mode. Note: In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the --driver-java-options command line option or in your default properties file.	3.0.0

Configuration	Default	Description	Since
spark.driver.extraJavaOptions	(none)	A string of extra JVM options to pass to the driver. This is intended to be set by users. For instance, GC settings or other logging. Note that it is illegal to set maximum heap size (-Xmx) settings with this option. Maximum heap size settings can be set with spark.driver.memory in the cluster mode and through the --driver-memory command line option in the client mode. Note: In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the --driver-java-options command line option or in your default properties file. spark.driver.defaultJavaOptions will be prepended to this configuration.	1.0.0
spark.driver.extraLibraryPath	(none)	Set a special library path to use when launching the driver JVM. Note: In client mode, this config must not be set through the SparkConf directly in your application, because the driver JVM has already started at that point. Instead, please set this through the --driver-library-path command line option or in your default properties file.	1.0.0
spark.driver.userClassPathFirst	false	(Experimental) Whether to give user-added jars precedence over Spark's own jars when loading classes in the driver. This feature can be used to mitigate conflicts between Spark's dependencies and user dependencies. It is currently an experimental feature. This is used in cluster mode only.	1.3.0
spark.executor.extraClassPath	(none)	Extra classpath entries to prepend to the classpath of executors. This exists primarily for backwards-compatibility with older versions of Spark. Users typically should not need to set this option.	1.0.0
spark.executor.defaultJavaOptions	(none)	A string of default JVM options to prepend to spark.executor.extraJavaOptions . This is intended to be set by administrators. For instance, GC settings or other logging. Note that it is illegal to set Spark properties or maximum heap size (-Xmx) settings with this option. Spark properties should be set using a SparkConf object or the spark-defaults.conf file used with the spark-submit script. Maximum heap size settings can be set with spark.executor.memory. The following symbols, if present will be interpolated: {{APP_ID}} will be replaced by application ID and {{EXECUTOR_ID}} will be replaced by executor ID. For example, to enable verbose gc logging to a file named for the executor ID of the app in /tmp, pass a 'value' of: -verbose:gc -Xloggc:/tmp/{{APP_ID}}-{{EXECUTOR_ID}}.gc	3.0.0
spark.executor.extraJavaOptions	(none)	A string of extra JVM options to pass to executors. This is intended to be set by users. For instance, GC settings or other logging. Note that it is illegal to set Spark properties or maximum heap size (-Xmx) settings with this option. Spark properties should be set using a SparkConf object or the spark-defaults.conf file used with the spark-submit script. Maximum heap size settings can be set with spark.executor.memory. The following symbols, if present will be interpolated: {{APP_ID}} will be replaced by application ID and {{EXECUTOR_ID}} will be replaced by executor ID. For example, to enable verbose gc logging to a file named for the executor ID of the app in /tmp, pass a 'value' of: -verbose:gc -Xloggc:/tmp/{{APP_ID}}-{{EXECUTOR_ID}}.gc spark.executor.defaultJavaOptions will be prepended to this configuration.	1.0.0
spark.executor.extraLibraryPath	(none)	Set a special library path to use when launching executor JVM's.	1.0.0
spark.executor.logs.rolling.maxRetainedFiles	-1	Sets the number of latest rolling log files that are going to be retained by the system. Older log files will be deleted. Disabled by default.	1.1.0
spark.executor.logs.rolling.enableCompression	false	Enable executor log compression. If it is enabled, the rolled executor logs will be compressed. Disabled by default.	2.0.2
spark.executor.logs.rolling.maxSize	1024 * 1024	Set the max size of the file in bytes by which the executor logs will be rolled over. Rolling is disabled by default. See spark.executor.logs.rolling.maxRetainedFiles for automatic cleaning of old logs.	1.4.0
spark.executor.logs.rolling.strategy	"" (disabled)	Set the strategy of rolling of executor logs. By default it is disabled. It can be set to "time" (time-based rolling) or "size" (size-based rolling) or "" (disabled). For "time", use spark.executor.logs.rolling.time.interval to set the rolling interval. For "size", use spark.executor.logs.rolling.maxSize to set the maximum file size for rolling.	1.1.0
spark.executor.logs.rolling.time.interval	daily	Set the time interval by which the executor logs will be rolled over. Rolling is disabled by default. Valid values are daily , hourly , minutely or any interval in seconds. See spark.executor.logs.rolling.maxRetainedFiles for automatic cleaning of old logs.	1.1.0
spark.executor.userClassPathFirst	false	(Experimental) Same functionality as spark.driver.userClassPathFirst , but applied to executor instances.	1.3.0
spark.executorEnv.[EnvironmentVariableName]	(none)	Add the environment variable specified by EnvironmentVariableName to the Executor process. The user can specify multiple of these to set multiple environment variables.	0.9.0
spark.redaction.regex	(?i)secret|password|token|access[.]?key	Regex to decide which Spark configuration properties and environment variables in driver and executor environments contain sensitive information. When this regex matches a property key or value, the value is redacted from the environment UI and various logs like YARN and event logs.	2.1.2
spark.redaction.string.regex	(none)	Regex to decide which parts of strings produced by Spark contain sensitive information. When this regex matches a string part, that string part is replaced by a dummy value. This is currently used to redact the output of SQL explain commands.	2.2.0
spark.python.profile	false	Enable profiling in Python worker, the profile result will show up by sc.show_profiles() , or it will be displayed before the driver exits. It also can be dumped into disk by sc.dump_profiles(path) . If some of the profile results had been displayed manually, they will not be displayed automatically before driver exiting. By default the pyspark.profiler.BasicProfiler will be used, but this can be overridden by passing a profiler class in as a parameter to the SparkContext constructor.	1.2.0
spark.python.profile.dump	(none)	The directory which is used to dump the profile result before driver exiting. The results will be dumped as separated file for each RDD. They can be loaded by pstats.Stats() . If this is specified, the profile result will not be displayed automatically.	1.2.0
spark.python.worker.memory	512m	Amount of memory to use per python worker process during aggregation, in the same format as JVM memory strings with a size unit suffix ("k", "m", "g" or "t") (e.g. 512m , 2g). If the memory used during aggregation goes above this amount, it will spill the data into disks.	1.1.0
spark.python.worker.reuse	true	Reuse Python worker or not. If yes, it will use a fixed number of Python workers, does not need to fork() a Python process for every task. It will be very useful if there is a large broadcast, then the broadcast will not need to be transferred from JVM to Python worker for every task.	1.2.0
spark.files		Comma-separated list of files to be placed in the working directory of each executor. Globs are allowed.	1.0.0
spark.submit.pyFiles		Comma-separated list of .zip, .egg, or .py files to place on the PYTHONPATH for Python apps. Globs are allowed.	1.0.1
spark.jars		Comma-separated list of jars to include on the driver and executor classpaths. Globs are allowed.	0.9.0
spark.jars.packages		Comma-separated list of Maven coordinates of jars to include on the driver and executor classpaths. The coordinates should be groupId:artifactId:version. If spark.jars.ivySettings is given artifacts will be resolved according to the configuration in the file, otherwise artifacts will be searched for in the local maven repo, then maven central and finally any additional remote repositories given by the command-line option --repositories . For more details, see Advanced Dependency Management .	1.5.0
spark.jars.excludes		Comma-separated list of groupId:artifactId, to exclude while resolving the dependencies provided in spark.jars.packages to avoid dependency conflicts.	1.5.0

Configuration	Default	Description	Since
spark.jars.ivy		Path to specify the Ivy user directory, used for the local Ivy cache and package files from spark.jars.packages . This will override the Ivy property ivy.default.ivy.user.dir which defaults to ~/.ivy2.	1.3.0
spark.jars.ivySettings		Path to an Ivy settings file to customize resolution of jars specified using spark.jars.packages instead of the built-in defaults, such as maven central. Additional repositories given by the command-line option --repositories or spark.jars.repositories will also be included. Useful for allowing Spark to resolve artifacts from behind a firewall e.g. via an in-house artifact server like Artifactory. Details on the settings file format can be found at Settings Files . Only paths with file:// scheme are supported. Paths without a scheme are assumed to have a file:// scheme. When running in YARN cluster mode, this file will also be localized to the remote driver for dependency resolution within SparkContext#addJar	2.2.0
spark.jars.repositories		Comma-separated list of additional remote repositories to search for the maven coordinates given with --packages or spark.jars.packages .	2.3.0
spark.archives		Comma-separated list of archives to be extracted into the working directory of each executor. .jar, .tar.gz, .tgz and .zip are supported. You can specify the directory name to unpack via adding # after the file name to unpack, for example, file.zip#directory . This configuration is experimental.	3.1.0
spark.pyspark.driver.python		Python binary executable to use for PySpark in driver. (default is spark.pyspark.python)	2.1.0
spark.pyspark.python		Python binary executable to use for PySpark in both driver and executors.	2.1.0
spark.reducer.maxSizeInFlight	48m	Maximum size of map outputs to fetch simultaneously from each reduce task, in MiB unless otherwise specified. Since each output requires us to create a buffer to receive it, this represents a fixed memory overhead per reduce task, so keep it small unless you have a large amount of memory.	1.4.0
spark.reducer.maxReqsInFlight	Int.MaxValue	This configuration limits the number of remote requests to fetch blocks at any given point. When the number of hosts in the cluster increase, it might lead to very large number of inbound connections to one or more nodes, causing the workers to fail under load. By allowing it to limit the number of fetch requests, this scenario can be mitigated.	2.0.0
spark.reducer.maxBlocksInFlightPerAddress	Int.MaxValue	This configuration limits the number of remote blocks being fetched per reduce task from a given host port. When a large number of blocks are being requested from a given address in a single fetch or simultaneously, this could crash the serving executor or Node Manager. This is especially useful to reduce the load on the Node Manager when external shuffle is enabled. You can mitigate this issue by setting it to a lower value.	2.2.1
spark.shuffle.compress	true	Whether to compress map output files. Generally a good idea. Compression will use spark.io.compression.codec .	0.6.0
spark.shuffle.file.buffer	32k	Size of the in-memory buffer for each shuffle file output stream, in KiB unless otherwise specified. These buffers reduce the number of disk seeks and system calls made in creating intermediate shuffle files.	1.4.0
spark.shuffle.file.merge.buffer	32k	Size of the in-memory buffer for each shuffle file input stream, in KiB unless otherwise specified. These buffers use off-heap buffers and are related to the number of files in the shuffle file. Too large buffers should be avoided.	4.0.0
spark.shuffle.unsafe.file.output.buffer	32k	Deprecated since Spark 4.0, please use spark.shuffle.localDisk.file.output.buffer .	2.3.0
spark.shuffle.localDisk.file.output.buffer	32k	The file system for this buffer size after each partition is written in all local disk shuffle writers. In KiB unless otherwise specified.	4.0.0
spark.shuffle.spill.diskWriteBufferSize	1024 * 1024	The buffer size, in bytes, to use when writing the sorted records to an on-disk file.	2.3.0
spark.shuffle.io.maxRetries	3	(Netty only) Fetches that fail due to IO-related exceptions are automatically retried if this is set to a non-zero value. This retry logic helps stabilize large shuffles in the face of long GC pauses or transient network connectivity issues.	1.2.0
spark.shuffle.io.numConnectionsPerPeer	1	(Netty only) Connections between hosts are reused in order to reduce connection buildup for large clusters. For clusters with many hard disks and few hosts, this may result in insufficient concurrency to saturate all disks, and so users may consider increasing this value.	1.2.1
spark.shuffle.io.preferDirectBufs	true	(Netty only) Off-heap buffers are used to reduce garbage collection during shuffle and cache block transfer. For environments where off-heap memory is tightly limited, users may wish to turn this off to force all allocations from Netty to be on-heap.	1.2.0
spark.shuffle.io.retryWait	5s	(Netty only) How long to wait between retries of fetches. The maximum delay caused by retrying is 15 seconds by default, calculated as maxRetries * retryWait .	1.2.1
spark.shuffle.io.backLog	-1	Length of the accept queue for the shuffle service. For large applications, this value may need to be increased, so that incoming connections are not dropped if the service cannot keep up with a large number of connections arriving in a short period of time. This needs to be configured wherever the shuffle service itself is running, which may be outside of the application (see spark.shuffle.service.enabled option below). If set below 1, will fallback to OS default defined by Netty's io.netty.util.NetUtil#SOMAXCONN .	1.1.1
spark.shuffle.io.connectionTimeout	value of spark.network.timeout	Timeout for the established connections between shuffle servers and clients to be marked as idled and closed if there are still outstanding fetch requests but no traffic no the channel for at least connectionTimeout .	1.2.0
spark.shuffle.io.connectionCreationTimeout	value of spark.shuffle.io.connectionTimeout	Timeout for establishing a connection between the shuffle servers and clients.	3.2.0
spark.shuffle.service.enabled	false	Enables the external shuffle service. This service preserves the shuffle files written by executors e.g. so that executors can be safely removed, or so that shuffle fetches can continue in the event of executor failure. The external shuffle service must be set up in order to enable it. See dynamic allocation configuration and setup documentation for more information.	1.2.0
spark.shuffle.service.port	7337	Port on which the external shuffle service will run.	1.2.0
spark.shuffle.service.name	spark_shuffle	The configured name of the Spark shuffle service the client should communicate with. This must match the name used to configure the Shuffle within the YARN NodeManager configuration (yarn.nodemanager.aux-services). Only takes effect when spark.shuffle.service.enabled is set to true.	3.2.0
spark.shuffle.service.index.cache.size	100m	Cache entries limited to the specified memory footprint, in bytes unless otherwise specified.	2.3.0
spark.shuffle.service.removeShuffle	true	Whether to use the ExternalShuffleService for deleting shuffle blocks for deallocated executors when the shuffle is no longer needed. Without this enabled, shuffle data on executors that are deallocated will remain on disk until the application ends.	3.3.0
spark.shuffle.maxChunksBeingTransferred	Long.MAX_VALUE	The max number of chunks allowed to be transferred at the same time on shuffle service. Note that new incoming connections will be closed when the max number is hit. The client will retry according to the shuffle retry configs (see spark.shuffle.io.maxRetries and spark.shuffle.io.retryWait), if those limits are reached the task will fail with fetch failure.	2.3.0

Configuration	Default	Description	Since
spark.shuffle.sort.bypassMergeThreshold	200	(Advanced) In the sort-based shuffle manager, avoid merge-sorting data if there is no map-side aggregation and there are at most this many reduce partitions.	1.1.1
spark.shuffle.sort.io.plugin.class	org.apache.spark.shuffle.sort.io.LocalDiskShuffleDataIO	Name of the class to use for shuffle IO.	3.0.0
spark.shuffle.spill.compress	true	Whether to compress data spilled during shuffles. Compression will use spark.io.compression.codec .	0.9.0
spark.shuffle.accurateBlockThreshold	100 * 1024 * 1024	Threshold in bytes above which the size of shuffle blocks in HighlyCompressedMapStatus is accurately recorded. This helps to prevent OOM by avoiding underestimating shuffle block size when fetch shuffle blocks.	2.2.1
spark.shuffle.accurateBlockSkewedFactor	-1.0	A shuffle block is considered as skewed and will be accurately recorded in HighlyCompressedMapStatus if its size is larger than this factor multiplying the median shuffle block size or spark.shuffle.accurateBlockThreshold . It is recommended to set this parameter to be the same as spark.sql.adaptive.skewJoin.skewedPartitionFactor . Set to -1.0 to disable this feature by default.	3.3.0
spark.shuffle.registration.timeout	5000	Timeout in milliseconds for registration to the external shuffle service.	2.3.0
spark.shuffle.registration.maxAttempts	3	When we fail to register to the external shuffle service, we will retry for maxAttempts times.	2.3.0
spark.shuffle.reduceLocality.enabled	true	Whether to compute locality preferences for reduce tasks.	1.5.0
spark.shuffle.mapOutput.minSizeForBroadcast	512k	The size at which we use Broadcast to send the map output statuses to the executors.	2.0.0
spark.shuffle.detectCorrupt	true	Whether to detect any corruption in fetched blocks.	2.2.0
spark.shuffle.detectCorrupt.useExtraMemory	false	If enabled, part of a compressed/encrypted stream will be de-compressed/de-crypted by using extra memory to detect early corruption. Any IOException thrown will cause the task to be retried once and if it fails again with same exception, then FetchFailedException will be thrown to retry previous stage.	3.0.0
spark.shuffle.useOldFetchProtocol	false	Whether to use the old protocol while doing the shuffle block fetching. It is only enabled while we need the compatibility in the scenario of new Spark version job fetching shuffle blocks from old version external shuffle service.	3.0.0
spark.shuffle.readHostLocalDisk	true	If enabled (and spark.shuffle.useOldFetchProtocol is disabled, shuffle blocks requested from those block managers which are running on the same host are read from the disk directly instead of being fetched as remote blocks over the network.	3.0.0
spark.files.io.connectionTimeout	value of spark.network.timeout	Timeout for the established connections for fetching files in Spark RPC environments to be marked as idled and closed if there are still outstanding files being downloaded but no traffic no the channel for at least connectionTimeout .	1.6.0
spark.files.io.connectionCreationTimeout	value of spark.files.io.connectionTimeout	Timeout for establishing a connection for fetching files in Spark RPC environments.	3.2.0
spark.shuffle.checksum.enabled	true	Whether to calculate the checksum of shuffle data. If enabled, Spark will calculate the checksum values for each partition data within the map output file and store the values in a checksum file on the disk. When there's shuffle data corruption detected, Spark will try to diagnose the cause (e.g., network issue, disk issue, etc.) of the corruption by using the checksum file.	3.2.0
spark.shuffle.checksum.algorithm	ADLER32	The algorithm is used to calculate the shuffle checksum. Currently, it only supports built-in algorithms of JDK, e.g., ADLER32, CRC32 and CRC32C.	3.2.0
spark.shuffle.service.fetch.rdd.enabled	false	Whether to use the ExternalShuffleService for fetching disk persisted RDD blocks. In case of dynamic allocation if this feature is enabled executors having only disk persisted blocks are considered idle after spark.dynamicAllocation.executorIdleTimeout and will be released accordingly.	3.0.0
spark.shuffle.service.db.enabled	true	Whether to use db in ExternalShuffleService. Note that this only affects standalone mode.	3.0.0
spark.shuffle.service.db.backend	ROCKSDB	Specifies a disk-based store used in shuffle service local db. Setting as ROCKSDB or LEVELDB (deprecated).	3.4.0
spark.eventLog.logBlockUpdates.enabled	false	Whether to log events for every block update, if spark.eventLog.enabled is true. *Warning*: This will increase the size of the event log considerably.	2.3.0
spark.eventLog.longForm.enabled	false	If true, use the long form of call sites in the event log. Otherwise use the short form.	2.4.0
spark.eventLog.compress	true	Whether to compress logged events, if spark.eventLog.enabled is true.	1.0.0
spark.eventLog.compression.codec	zstd	The codec to compress logged events. By default, Spark provides four codecs: lz4 , lzf , snappy , and zstd . You can also use fully qualified class names to specify the codec, e.g. org.apache.spark.io.LZ4CompressionCodec , org.apache.spark.io.LZFCompressionCodec , org.apache.spark.io.SnappyCompressionCodec , and org.apache.spark.io.ZStdCompressionCodec .	3.0.0
spark.eventLog.erasureCoding.enabled	false	Whether to allow event logs to use erasure coding, or turn erasure coding off, regardless of filesystem defaults. On HDFS, erasure coded files will not update as quickly as regular replicated files, so the application updates will take longer to appear in the History Server. Note that even if this is true, Spark will still not force the file to use erasure coding, it will simply use filesystem defaults.	3.0.0
spark.eventLog.dir	file:///tmp/spark-events	Base directory in which Spark events are logged, if spark.eventLog.enabled is true. Within this base directory, Spark creates a sub-directory for each application, and logs the events specific to the application in this directory. Users may want to set this to a unified location like an HDFS directory so history files can be read by the history server.	1.0.0
spark.eventLog.enabled	false	Whether to log Spark events, useful for reconstructing the Web UI after the application has finished.	1.0.0
spark.eventLog.overwrite	false	Whether to overwrite any existing files.	1.0.0
spark.eventLog.buffer.kb	100k	Buffer size to use when writing to output streams, in KiB unless otherwise specified.	1.0.0
spark.eventLog.rolling.enabled	false	Whether rolling over event log files is enabled. If set to true, it cuts down each event log file to the configured size.	3.0.0
spark.eventLog.rolling.maxFileSize	128m	When spark.eventLog.rolling.enabled=true , specifies the max size of event log file before it's rolled over.	3.0.0
spark.ui.dagGraph.retainedRootRDDs	Int.MaxValue	How many DAG graph nodes the Spark UI and status APIs remember before garbage collecting.	2.1.0
spark.ui.groupSQLSubExecutionEnabled	true	Whether to group sub executions together in SQL UI when they belong to the same root execution	3.4.0

Configuration	Default	Description	Since
spark.ui.enabled	true	Whether to run the web UI for the Spark application.	1.1.1
spark.ui.store.path	None	Local directory where to cache application information for live UI. By default this is not set, meaning all application information will be kept in memory.	3.4.0
spark.ui.killEnabled	true	Allows jobs and stages to be killed from the web UI.	1.0.0
spark.ui.threadDumpsEnabled	true	Whether to show a link for executor thread dumps in Stages and Executor pages.	1.2.0
spark.ui.threadDump.flamegraphEnabled	true	Whether to render the Flamegraph for executor thread dumps.	4.0.0
spark.ui.heapHistogramEnabled	true	Whether to show a link for executor heap histogram in Executor page.	3.5.0
spark.ui.liveUpdate.period	100ms	How often to update live entities. -1 means "never update" when replaying applications, meaning only the last write will happen. For live applications, this avoids a few operations that we can live without when rapidly processing incoming task events.	2.3.0
spark.ui.liveUpdate.minFlushPeriod	1s	Minimum time elapsed before stale UI data is flushed. This avoids UI staleness when incoming task events are not fired frequently.	2.4.2
spark.ui.port	4040	Port for your application's dashboard, which shows memory and workload data.	0.7.0
spark.ui.retainedJobs	1000	How many jobs the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	1.2.0
spark.ui.retainedStages	1000	How many stages the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	0.9.0
spark.ui.retainedTasks	100000	How many tasks in one stage the Spark UI and status APIs remember before garbage collecting. This is a target maximum, and fewer elements may be retained in some circumstances.	2.0.1
spark.ui.reverseProxy	false	Enable running Spark Master as reverse proxy for worker and application UIs. In this mode, Spark master will reverse proxy the worker and application UIs to enable access without requiring direct access to their hosts. Use it with caution, as worker and application UI will not be accessible directly, you will only be able to access them through spark master/proxy public URL. This setting affects all the workers and application UIs running in the cluster and must be set on all the workers, drivers and masters.	2.1.0
spark.ui.reverseProxyUrl		If the Spark UI should be served through another front-end reverse proxy, this is the URL for accessing the Spark master UI through that reverse proxy. This is useful when running proxy for authentication e.g. an OAuth proxy. The URL may contain a path prefix, like http://mydomain.com/path/to/spark/ , allowing you to serve the UI for multiple Spark clusters and other web applications through the same virtual host and port. Normally, this should be an absolute URL including scheme (http/https), host and port. It is possible to specify a relative URL starting with "/" here. In this case, all URLs generated by the Spark UI and Spark REST APIs will be server-relative links -- this will still work, as the entire Spark UI is served through the same host and port. The setting affects link generation in the Spark UI, but the front-end reverse proxy is responsible for stripping a path prefix before forwarding the request, rewriting redirects which point directly to the Spark master, redirecting access from http://mydomain.com/path/to/spark to http://mydomain.com/path/to/spark/ (trailing slash after path prefix); otherwise relative links on the master page do not work correctly. This setting affects all the workers and application UIs running in the cluster and must be set identically on all the workers, drivers and masters. In is only effective when spark.ui.reverseProxy is turned on. This setting is not needed when the Spark master web UI is directly reachable. Note that the value of the setting can't contain the keyword proxy or history after split by "/". Spark UI relies on both keywords for getting REST API endpoints from URIs.	2.1.0
spark.ui.proxyRedirectUri		Where to address redirects when Spark is running behind a proxy. This will make Spark modify redirect responses so they point to the proxy server, instead of the Spark UI's own address. This should be only the address of the server, without any prefix paths for the application; the prefix should be set either by the proxy server itself (by adding the X-Forwarded-Context request header), or by setting the proxy base in the Spark app's configuration.	3.0.0
spark.ui.showConsoleProgress	false	Show the progress bar in the console. The progress bar shows the progress of stages that run for longer than 500ms. If multiple stages run at the same time, multiple progress bars will be displayed on the same line. Note: In shell environment, the default value of spark.ui.showConsoleProgress is true.	1.2.1
spark.ui.consoleProgress.update.interval	200	An interval in milliseconds to update the progress bar in the console.	2.1.0
spark.ui.custom.executor.log.url	(none)	Specifies custom spark executor log URL for supporting external log service instead of using cluster managers' application log URLs in Spark UI. Spark will support some path variables via patterns which can vary on cluster manager. Please check the documentation for your cluster manager to see which patterns are supported, if any. Please note that this configuration also replaces original log urls in event log, which will be also effective when accessing the application on history server. The new log urls must be permanent, otherwise you might have dead link for executor log urls. For now, only YARN and K8s cluster manager supports this configuration	3.0.0
spark.ui.prometheus.enabled	true	Expose executor metrics at /metrics/executors/prometheus at driver web page.	3.0.0
spark.worker.ui.retainedExecutors	1000	How many finished executors the Spark UI and status APIs remember before garbage collecting.	1.5.0
spark.worker.ui.retainedDrivers	1000	How many finished drivers the Spark UI and status APIs remember before garbage collecting.	1.5.0
spark.sql.ui.retainedExecutions	1000	How many finished executions the Spark UI and status APIs remember before garbage collecting.	1.5.0
spark.streaming.ui.retainedBatches	1000	How many finished batches the Spark UI and status APIs remember before garbage collecting.	1.0.0
spark.ui.retainedDeadExecutors	100	How many dead executors the Spark UI and status APIs remember before garbage collecting.	2.0.0
spark.ui.filters	None	Comma separated list of filter class names to apply to the Spark Web UI. The filter should be a standard javax servlet Filter . Filter parameters can also be specified in the configuration, by setting config entries of the form spark..param.= For example: spark.ui.filters=com.test.filter1 spark.com.test.filter1.param.name1=foo spark.com.test.filter1.param.name2=bar	1.0.0
spark.ui.requestHeaderSize	8k	The maximum allowed size for a HTTP request header, in bytes unless otherwise specified. This setting applies for the Spark History Server too.	2.2.3
spark.ui.timelineEnabled	true	Whether to display event timeline data on UI pages.	3.4.0
spark.ui.timeline.executors.maximum	250	The maximum number of executors shown in the event timeline.	3.2.0
spark.ui.timeline.jobs.maximum	500	The maximum number of jobs shown in the event timeline.	3.2.0
spark.ui.timeline.stages.maximum	500	The maximum number of stages shown in the event timeline.	3.2.0

Configuration	Default	Description	Since
spark.ui.timeline.tasks.maximum	1000	The maximum number of tasks shown in the event timeline.	1.4.0
spark.appStatusStore.diskStoreDir	None	Local directory where to store diagnostic information of SQL executions. This configuration is only for live UI.	3.4.0
spark.broadcast.compress	true	Whether to compress broadcast variables before sending them. Generally a good idea. Compression will use spark.io.compression.codec .	0.6.0
spark.checkpoint.dir	(none)	Set the default directory for checkpointing. It can be overwritten by SparkContext.setCheckpointDir.	4.0.0
spark.checkpoint.compress	false	Whether to compress RDD checkpoints. Generally a good idea. Compression will use spark.io.compression.codec .	2.2.0
spark.io.compression.codec	lz4	The codec used to compress internal data such as RDD partitions, event log, broadcast variables and shuffle outputs. By default, Spark provides four codecs: lz4 , lzf , snappy , and zstd . You can also use fully qualified class names to specify the codec, e.g. org.apache.spark.io.LZ4CompressionCodec , org.apache.spark.io.LZFCompressionCodec , org.apache.spark.io.SnappyCompressionCodec , and org.apache.spark.io.ZStdCompressionCodec .	0.8.0
spark.io.compression.lz4.blockSize	32k	Block size used in LZ4 compression, in the case when LZ4 compression codec is used. Lowering this block size will also lower shuffle memory usage when LZ4 is used. Default unit is bytes, unless otherwise specified. This configuration only applies to spark.io.compression.codec .	1.4.0
spark.io.compression.snappy.blockSize	32k	Block size in Snappy compression, in the case when Snappy compression codec is used. Lowering this block size will also lower shuffle memory usage when Snappy is used. Default unit is bytes, unless otherwise specified. This configuration only applies to spark.io.compression.codec .	1.4.0
spark.io.compression.zstd.level	1	Compression level for Zstd compression codec. Increasing the compression level will result in better compression at the expense of more CPU and memory. This configuration only applies to spark.io.compression.codec .	2.3.0
spark.io.compression.zstd.bufferSize	32k	Buffer size in bytes used in Zstd compression, in the case when Zstd compression codec is used. Lowering this size will lower the shuffle memory usage when Zstd is used, but it might increase the compression cost because of excessive JNI call overhead. This configuration only applies to spark.io.compression.codec .	2.3.0
spark.io.compression.zstd.bufferPool.enabled	true	If true, enable buffer pool of ZSTD JNI library.	3.2.0
spark.io.compression.zstd.workers	0	Thread size spawned to compress in parallel when using Zstd. When value is 0 no worker is spawned, it works in single-threaded mode. When value > 0, it triggers asynchronous mode, corresponding number of threads are spawned. More workers improve performance, but also increase memory cost.	4.0.0
spark.io.compression.lzf.parallel.enabled	false	When true, LZF compression will use multiple threads to compress data in parallel.	4.0.0
spark.kryo.classesToRegister	(none)	If you use Kryo serialization, give a comma-separated list of custom class names to register with Kryo. See the tuning guide for more details.	1.2.0
spark.kryo.referenceTracking	true	Whether to track references to the same object when serializing data with Kryo, which is necessary if your object graphs have loops and useful for efficiency if they contain multiple copies of the same object. Can be disabled to improve performance if you know this is not the case.	0.8.0
spark.kryo.registrationRequired	false	Whether to require registration with Kryo. If set to 'true', Kryo will throw an exception if an unregistered class is serialized. If set to false (the default), Kryo will write unregistered class names along with each object. Writing class names can cause significant performance overhead, so enabling this option can enforce strictly that a user has not omitted classes from registration.	1.1.0
spark.kryo.registrator	(none)	If you use Kryo serialization, give a comma-separated list of classes that register your custom classes with Kryo. This property is useful if you need to register your classes in a custom way, e.g. to specify a custom field serializer. Otherwise spark.kryo.classesToRegister is simpler. It should be set to classes that extend KryoRegistrator . See the tuning guide for more details.	0.5.0
spark.kryo.unsafe	true	Whether to use unsafe based Kryo serializer. Can be substantially faster by using Unsafe Based IO.	2.1.0
spark.kryoserializer.buffer.max	64m	Maximum allowable size of Kryo serialization buffer, in MiB unless otherwise specified. This must be larger than any object you attempt to serialize and must be less than 2048m. Increase this if you get a "buffer limit exceeded" exception inside Kryo.	1.4.0
spark.kryoserializer.buffer	64k	Initial size of Kryo's serialization buffer, in KiB unless otherwise specified. Note that there will be one buffer per core on each worker. This buffer will grow up to spark.kryoserializer.buffer.max if needed.	1.4.0
spark.rdd.compress	false	Whether to compress serialized RDD partitions (e.g. for StorageLevel.MEMORY_ONLY_SER in Java and Scala or StorageLevel.MEMORY_ONLY in Python). Can save substantial space at the cost of some extra CPU time. Compression will use spark.io.compression.codec .	0.6.0
spark.serializer	org.apache.spark.serializer. JavaSerializer	Class to use for serializing objects that will be sent over the network or need to be cached in serialized form. The default of Java serialization works with any Serializable Java object but is quite slow, so we recommend using org.apache.spark.serializer.KryoSerializer and configuring Kryo serialization when speed is necessary. Can be any subclass of org.apache.spark.Serializer .	0.5.0
spark.serializer.objectStreamReset	100	When serializing using org.apache.spark.serializer.JavaSerializer, the serializer caches objects to prevent writing redundant data, however that stops garbage collection of those objects. By calling 'reset' you flush that info from the serializer, and allow old objects to be collected. To turn off this periodic reset set it to -1. By default it will reset the serializer every 100 objects.	1.0.0
spark.memory.fraction	0.6	Fraction of (heap space - 300MB) used for execution and storage. The lower this is, the more frequently spills and cached data eviction occur. The purpose of this config is to set aside memory for internal metadata, user data structures, and imprecise size estimation in the case of sparse, unusually large records. Leaving this at the default value is recommended. For more detail, including important information about correctly tuning JVM garbage collection when increasing this value, see this description .	1.6.0
spark.memory.storageFraction	0.5	Amount of storage memory immune to eviction, expressed as a fraction of the size of the region set aside by spark.memory.fraction . The higher this is, the less working memory may be available to execution and tasks may spill to disk more often. Leaving this at the default value is recommended. For more detail, see this description .	1.6.0
spark.memory.offHeap.enabled	false	If true, Spark will attempt to use off-heap memory for certain operations. If off-heap memory use is enabled, then spark.memory.offHeap.size must be positive.	1.6.0
spark.memory.offHeap.size	0	The absolute amount of memory which can be used for off-heap allocation, in bytes unless otherwise specified. This setting has no impact on heap memory usage, so if your executors' total memory consumption must fit within some hard limit then be sure to shrink your JVM heap size accordingly. This must be set to a positive value when spark.memory.offHeap.enabled=true .	1.6.0
spark.storage.unrollMemoryThreshold	1024 * 1024	Initial memory to request before unrolling any block.	1.1.0

Configuration	Default	Description	Since
spark.storage.replication.proactive	true	Enables proactive block replication for RDD blocks. Cached RDD block replicas lost due to executor failures are replenished if there are any existing available replicas. This tries to get the replication level of the block to the initial number.	2.2.0
spark.storage.localDiskByExecutors.cacheSize	1000	The max number of executors for which the local dirs are stored. This size is both applied for the driver and both for the executors side to avoid having an unbounded store. This cache will be used to avoid the network in case of fetching disk persisted RDD blocks or shuffle blocks (when spark.shuffle.readHostLocalDisk is set) from the same host.	3.0.0
spark.cleaner.periodicGC.interval	30min	Controls how often to trigger a garbage collection. This context cleaner triggers cleanups only when weak references are garbage collected. In long-running applications with large driver JVMs, where there is little memory pressure on the driver, this may happen very occasionally or not at all. Not cleaning at all may lead to executors running out of disk space after a while.	1.6.0
spark.cleaner.referenceTracking	true	Enables or disables context cleaning.	1.0.0
spark.cleaner.referenceTracking.blocking	true	Controls whether the cleaning thread should block on cleanup tasks (other than shuffle, which is controlled by spark.cleaner.referenceTracking.blocking.shuffle Spark property).	1.0.0
spark.cleaner.referenceTracking.blocking.shuffle	false	Controls whether the cleaning thread should block on shuffle cleanup tasks.	1.1.1
spark.cleaner.referenceTracking.cleanCheckpoints	false	Controls whether to clean checkpoint files if the reference is out of scope.	1.4.0
spark.broadcast.blockSize	4m	Size of each piece of a block for TorrentBroadcastFactory , in KiB unless otherwise specified. Too large a value decreases parallelism during broadcast (makes it slower); however, if it is too small, BlockManager might take a performance hit.	0.5.0
spark.broadcast.checksum	true	Whether to enable checksum for broadcast. If enabled, broadcasts will include a checksum, which can help detect corrupted blocks, at the cost of computing and sending a little more data. It's possible to disable it if the network has other mechanisms to guarantee data won't be corrupted during broadcast.	2.1.1
spark.broadcast.UDFCompressionThreshold	1 * 1024 * 1024	The threshold at which user-defined functions (UDFs) and Python RDD commands are compressed by broadcast in bytes unless otherwise specified.	3.0.0
spark.executor.cores	1 in YARN mode, all the available cores on the worker in standalone mode.	The number of cores to use on each executor.	1.0.0
spark.default.parallelism	For distributed shuffle operations like reduceByKey and join , the largest number of partitions in a parent RDD. For operations like parallelize with no parent RDDs, it depends on the cluster manager: Local mode: number of cores on the local machine Others: total number of cores on all executor nodes or 2, whichever is larger	Default number of partitions in RDDs returned by transformations like join , reduceByKey , and parallelize when not set by user.	0.5.0
spark.executor.heartbeatInterval	10s	Interval between each executor's heartbeats to the driver. Heartbeats let the driver know that the executor is still alive and update it with metrics for in-progress tasks. spark.executor.heartbeatInterval should be significantly less than spark.network.timeout	1.1.0
spark.files.fetchTimeout	60s	Communication timeout to use when fetching files added through SparkContext.addFile() from the driver.	1.0.0
spark.files.useFetchCache	true	If set to true (default), file fetching will use a local cache that is shared by executors that belong to the same application, which can improve task launching performance when running many executors on the same host. If set to false, these caching optimizations will be disabled and all executors will fetch their own copies of files. This optimization may be disabled in order to use Spark local directories that reside on NFS filesystems (see SPARK-6313 for more details).	1.2.2
spark.files.overwrite	false	Whether to overwrite any files which exist at the startup. Users can not overwrite the files added by SparkContext.addFile or SparkContext.addJar before even if this option is set true .	1.0.0
spark.files.ignoreCorruptFiles	false	Whether to ignore corrupt files. If true, the Spark jobs will continue to run when encountering corrupted or non-existing files and contents that have been read will still be returned.	2.1.0
spark.files.ignoreMissingFiles	false	Whether to ignore missing files. If true, the Spark jobs will continue to run when encountering missing files and the contents that have been read will still be returned.	2.4.0
spark.files.maxPartitionBytes	134217728 (128 MiB)	The maximum number of bytes to pack into a single partition when reading files.	2.1.0
spark.files.openCostInBytes	4194304 (4 MiB)	The estimated cost to open a file, measured by the number of bytes could be scanned at the same time. This is used when putting multiple files into a partition. It is better to overestimate, then the partitions with small files will be faster than partitions with bigger files.	2.1.0
spark.hadoop.cloneConf	false	If set to true, clones a new Hadoop Configuration object for each task. This option should be enabled to work around Configuration thread-safety issues (see SPARK-2546 for more details). This is disabled by default in order to avoid unexpected performance regressions for jobs that are not affected by these issues.	1.0.3
spark.hadoop.validateOutputSpecs	true	If set to true, validates the output specification (e.g. checking if the output directory already exists) used in saveAsHadoopFile and other variants. This can be disabled to silence exceptions due to pre-existing output directories. We recommend that users do not disable this except if trying to achieve compatibility with previous versions of Spark. Simply use Hadoop's FileSystem API to delete output directories by hand. This setting is ignored for jobs generated through Spark Streaming's StreamingContext, since data may need to be rewritten to pre-existing output directories during checkpoint recovery.	1.0.1
spark.storage.memoryMapThreshold	2m	Size of a block above which Spark memory maps when reading a block from disk. Default unit is bytes, unless specified otherwise. This prevents Spark from memory mapping very small blocks. In general, memory mapping has high overhead for blocks close to or below the page size of the operating system.	0.9.2
spark.storage.decommission.enabled	false	Whether to decommission the block manager when decommissioning executor.	3.1.0
spark.storage.decommission.shuffleBlocks.enabled	true	Whether to transfer shuffle blocks during block manager decommissioning. Requires a migratable shuffle resolver (like sort based shuffle).	3.1.0

Configuration	Default	Description	Since
spark.storage.decommission.shuffleBlocks.maxThreads	8	Maximum number of threads to use in migrating shuffle files.	3.1.0
spark.storage.decommission.rddBlocks.enabled	true	Whether to transfer RDD blocks during block manager decommissioning.	3.1.0
spark.storage.decommission.fallbackStorage.path	(none)	The location for fallback storage during block manager decommissioning. For example, s3a://spark-storage/ . In case of empty, fallback storage is disabled. The storage should be managed by TTL because Spark will not clean it up.	3.1.0
spark.storage.decommission.fallbackStorage.cleanUp	false	If true, Spark cleans up its fallback storage data during shutting down.	3.2.0
spark.storage.decommission.shuffleBlocks.maxDiskSize	(none)	Maximum disk space to use to store shuffle blocks before rejecting remote shuffle blocks. Rejecting remote shuffle blocks means that an executor will not receive any shuffle migrations, and if there are no other executors available for migration then shuffle blocks will be lost unless spark.storage.decommission.fallbackStorage.path is configured.	3.2.0
spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version	1	The file output committer algorithm version, valid algorithm version number: 1 or 2. Note that 2 may cause a correctness issue like MAPREDUCE-7282.	2.2.0
spark.eventLog.logStageExecutorMetrics	false	Whether to write per-stage peaks of executor metrics (for each executor) to the event log. Note: The metrics are polled (collected) and sent in the executor heartbeat, and this is always done; this configuration is only to determine if aggregated metric peaks are written to the event log.	3.0.0
spark.executor.processTreeMetrics.enabled	false	Whether to collect process tree metrics (from the /proc filesystem) when collecting executor metrics. Note: The process tree metrics are collected only if the /proc filesystem exists.	3.0.0
spark.executor.metrics.pollingInterval	0	How often to collect executor metrics (in milliseconds). If 0, the polling is done on executor heartbeats (thus at the heartbeat interval, specified by spark.executor.heartbeatInterval). If positive, the polling is done at this interval.	3.0.0
spark.eventLog.gcMetrics.youngGenerationGarbageCollectors	Copy,PS Scavenge,ParNew,G1 Young Generation	Names of supported young generation garbage collector. A name usually is the return of GarbageCollectorMXBean.getName. The built-in young generation garbage collectors are Copy,PS Scavenge,ParNew,G1 Young Generation.	3.0.0
spark.eventLog.gcMetrics.oldGenerationGarbageCollectors	MarkSweepCompact,PS MarkSweep,ConcurrentMarkSweep,G1 Old Generation	Names of supported old generation garbage collector. A name usually is the return of GarbageCollectorMXBean.getName. The built-in old generation garbage collectors are MarkSweepCompact,PS MarkSweep,ConcurrentMarkSweep,G1 Old Generation.	3.0.0
spark.executor.metrics.fileSystemSchemes	file,hdfs	The file system schemes to report in executor metrics.	3.1.0
spark.rpc.message.maxSize	128	Maximum message size (in MiB) to allow in "control plane" communication; generally only applies to map output size information sent between executors and the driver. Increase this if you are running jobs with many thousands of map and reduce tasks and see messages about the RPC message size.	2.0.0
spark.blockManager.port	(random)	Port for all block managers to listen on. These exist on both the driver and the executors.	1.1.0
spark.driver.blockManager.port	(value of spark.blockManager.port)	Driver-specific port for the block manager to listen on, for cases where it cannot use the same configuration as executors.	2.1.0
spark.driver.bindAddress	(value of spark.driver.host)	Hostname or IP address where to bind listening sockets. This config overrides the SPARK_LOCAL_IP environment variable (see below). It also allows a different address from the local one to be advertised to executors or external systems. This is useful, for example, when running containers with bridged networking. For this to properly work, the different ports used by the driver (RPC, block manager and UI) need to be forwarded from the container's host.	2.1.0
spark.driver.host	(local hostname)	Hostname or IP address for the driver. This is used for communicating with the executors and the standalone Master.	0.7.0
spark.driver.port	(random)	Port for the driver to listen on. This is used for communicating with the executors and the standalone Master.	0.7.0
spark.rpc.io.backLog	64	Length of the accept queue for the RPC server. For large applications, this value may need to be increased, so that incoming connections are not dropped when a large number of connections arrives in a short period of time.	3.0.0
spark.network.timeout	120s	Default timeout for all network interactions. This config will be used in place of spark.storage.blockManagerHeartbeatTimeoutMs , spark.shuffle.io.connectionTimeout , spark.rpc.askTimeout or spark.rpc.lookupTimeout if they are not configured.	1.3.0
spark.network.timeoutInterval	60s	Interval for the driver to check and expire dead executors.	1.3.2
spark.network.io.preferDirectBufs	true	If enabled then off-heap buffer allocations are preferred by the shared allocators. Off-heap buffers are used to reduce garbage collection during shuffle and cache block transfer. For environments where off-heap memory is tightly limited, users may wish to turn this off to force all allocations to be on-heap.	3.0.0
spark.port.maxRetries	16	Maximum number of retries when binding to a port before giving up. When a port is given a specific value (non 0), each subsequent retry will increment the port used in the previous attempt by 1 before retrying. This essentially allows it to try a range of ports from the start port specified to port + maxRetries.	1.1.1
spark.rpc.askTimeout	spark.network.timeout	Duration for an RPC ask operation to wait before timing out.	1.4.0
spark.rpc.lookupTimeout	120s	Duration for an RPC remote endpoint lookup operation to wait before timing out.	1.4.0
spark.network.maxRemoteBlockSizeFetchToMem	200m	Remote block will be fetched to disk when size of the block is above this threshold in bytes. This is to avoid a giant request takes too much memory. Note this configuration will affect both shuffle fetch and block manager remote block fetch. For users who enabled external shuffle service, this feature can only work when external shuffle service is at least 2.3.0.	3.0.0
spark.rpc.io.connectionTimeout	value of spark.network.timeout	Timeout for the established connections between RPC peers to be marked as idled and closed if there are outstanding RPC requests but no traffic on the channel for at least connectionTimeout .	1.2.0
spark.rpc.io.connectionCreationTimeout	value of spark.rpc.io.connectionTimeout	Timeout for establishing a connection between RPC peers.	3.2.0
spark.cores.max	(not set)	When running on a standalone deploy cluster , the maximum amount of CPU cores to request for the application from across the cluster (not from each machine). If not set, the default will be spark.deploy.defaultCores on Spark's standalone cluster manager.	0.6.0

Configuration	Default	Description	Since
spark.locality.wait	3s	How long to wait to launch a data-local task before giving up and launching it on a less-local node. The same wait will be used to step through multiple locality levels (process-local, node-local, rack-local and then any). It is also possible to customize the waiting time for each level by setting spark.locality.wait.node , etc. You should increase this setting if your tasks are long and see poor locality, but the default usually works well.	0.5.0
spark.locality.wait.node	spark.locality.wait	Customize the locality wait for node locality. For example, you can set this to 0 to skip node locality and search immediately for rack locality (if your cluster has rack information).	0.8.0
spark.locality.wait.process	spark.locality.wait	Customize the locality wait for process locality. This affects tasks that attempt to access cached data in a particular executor process.	0.8.0
spark.locality.wait.rack	spark.locality.wait	Customize the locality wait for rack locality.	0.8.0
spark.scheduler.maxRegisteredResourcesWaitingTime	30s	Maximum amount of time to wait for resources to register before scheduling begins.	1.1.1
spark.scheduler.minRegisteredResourcesRatio	0.8 for KUBERNETES mode; 0.8 for YARN mode; 0.0 for standalone mode	The minimum ratio of registered resources (registered resources / total expected resources) (resources are executors in yarn mode and Kubernetes mode, CPU cores in standalone mode) to wait for before scheduling begins. Specified as a double between 0.0 and 1.0. Regardless of whether the minimum ratio of resources has been reached, the maximum amount of time it will wait before scheduling begins is controlled by config spark.scheduler.maxRegisteredResourcesWaitingTime .	1.1.1
spark.scheduler.mode	FIFO	The scheduling mode between jobs submitted to the same SparkContext. Can be set to FAIR to use fair sharing instead of queueing jobs one after another. Useful for multi-user services.	0.8.0
spark.scheduler.revive.interval	1s	The interval length for the scheduler to revive the worker resource offers to run tasks.	0.8.1
spark.scheduler.listenerbus.eventqueue.capacity	10000	The default capacity for event queues. Spark will try to initialize an event queue using capacity specified by spark.scheduler.listenerbus.eventqueue.queueName.capacity first. If it's not configured, Spark will use the default capacity specified by this config. Note that capacity must be greater than 0. Consider increasing value (e.g. 20000) if listener events are dropped. Increasing this value may result in the driver using more memory.	2.3.0
spark.scheduler.listenerbus.eventqueue.shared.capacity	spark.scheduler.listenerbus.eventqueue.capacity	Capacity for shared event queue in Spark listener bus, which hold events for external listener(s) that register to the listener bus. Consider increasing value, if the listener events corresponding to shared queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
spark.scheduler.listenerbus.eventqueue.appStatus.capacity	spark.scheduler.listenerbus.eventqueue.capacity	Capacity for appStatus event queue, which hold events for internal application status listeners. Consider increasing value, if the listener events corresponding to appStatus queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
spark.scheduler.listenerbus.eventqueue.executorManagement.capacity	spark.scheduler.listenerbus.eventqueue.capacity	Capacity for executorManagement event queue in Spark listener bus, which hold events for internal executor management listeners. Consider increasing value if the listener events corresponding to executorManagement queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
spark.scheduler.listenerbus.eventqueue.eventLog.capacity	spark.scheduler.listenerbus.eventqueue.capacity	Capacity for eventLog queue in Spark listener bus, which hold events for Event logging listeners that write events to eventLogs. Consider increasing value if the listener events corresponding to eventLog queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
spark.scheduler.listenerbus.eventqueue.streams.capacity	spark.scheduler.listenerbus.eventqueue.capacity	Capacity for streams queue in Spark listener bus, which hold events for internal streaming listener. Consider increasing value if the listener events corresponding to streams queue are dropped. Increasing this value may result in the driver using more memory.	3.0.0
spark.scheduler.resource.profileMergeConflicts	false	If set to "true", Spark will merge ResourceProfiles when different profiles are specified in RDDs that get combined into a single stage. When they are merged, Spark chooses the maximum of each resource and creates a new ResourceProfile. The default of false results in Spark throwing an exception if multiple different ResourceProfiles are found in RDDs going into the same stage.	3.1.0
spark.scheduler.excludeOnFailure.unschedulableTaskSetTimeout	120s	The timeout in seconds to wait to acquire a new executor and schedule a task before aborting a TaskSet which is unschedulable because all executors are excluded due to task failures.	2.4.1
spark.standalone.submit.waitAppCompletion	false	If set to true, Spark will merge ResourceProfiles when different profiles are specified in RDDs that get combined into a single stage. When they are merged, Spark chooses the maximum of each resource and creates a new ResourceProfile. The default of false results in Spark throwing an exception if multiple different ResourceProfiles are found in RDDs going into the same stage.	3.1.0
spark.excludeOnFailure.enabled	false	If set to "true", prevent Spark from scheduling tasks on executors that have been excluded due to too many task failures. The algorithm used to exclude executors and nodes can be further controlled by the other "spark.excludeOnFailure" configuration options. This config will be overridden by "spark.excludeOnFailure.application.enabled" and "spark.excludeOnFailure.taskAndStage.enabled" to specify exclusion enablement on individual levels.	2.1.0
spark.excludeOnFailure.application.enabled	false	If set to "true", enables excluding executors for the entire application due to too many task failures and prevent Spark from scheduling tasks on them. This config overrides "spark.excludeOnFailure.enabled".	4.0.0
spark.excludeOnFailure.taskAndStage.enabled	false	If set to "true", enables excluding executors on a task set level due to too many task failures and prevent Spark from scheduling tasks on them. This config overrides "spark.excludeOnFailure.enabled".	4.0.0
spark.excludeOnFailure.timeout	1h	(Experimental) How long a node or executor is excluded for the entire application, before it is unconditionally removed from the excludelist to attempt running new tasks.	2.1.0
spark.excludeOnFailure.task.maxTaskAttemptsPerExecutor	1	(Experimental) For a given task, how many times it can be retried on one executor before the executor is excluded for that task.	2.1.0
spark.excludeOnFailure.task.maxTaskAttemptsPerNode	2	(Experimental) For a given task, how many times it can be retried on one node, before the entire node is excluded for that task.	2.1.0
spark.excludeOnFailure.stage.maxFailedTasksPerExecutor	2	(Experimental) How many different tasks must fail on one executor, within one stage, before the executor is excluded for that stage.	2.1.0
spark.excludeOnFailure.stage.maxFailedExecutorsPerNode	2	(Experimental) How many different executors are marked as excluded for a given stage, before the entire node is marked as failed for the stage.	2.1.0

Configuration	Default	Description	Since
spark.excludeOnFailure.application.maxFailedTasksPerExecutor	2	(Experimental) How many different tasks must fail on one executor, in successful task sets, before the executor is excluded for the entire application. Excluded executors will be automatically added back to the pool of available resources after the timeout specified by spark.excludeOnFailure.timeout . Note that with dynamic allocation, though, the executors may get marked as idle and be reclaimed by the cluster manager.	2.2.0
spark.excludeOnFailure.application.maxFailedExecutorsPerNode	2	(Experimental) How many different executors must be excluded for the entire application, before the node is excluded for the entire application. Excluded nodes will be automatically added back to the pool of available resources after the timeout specified by spark.excludeOnFailure.timeout . Note that with dynamic allocation, though, the executors on the node may get marked as idle and be reclaimed by the cluster manager.	2.2.0
spark.excludeOnFailure.killExcludedExecutors	false	(Experimental) If set to "true", allow Spark to automatically kill the executors when they are excluded on fetch failure or excluded for the entire application, as controlled by spark.killExcludedExecutors.application.*. Note that, when an entire node is added excluded, all of the executors on that node will be killed.	2.2.0
spark.excludeOnFailure.application.fetchFailure.enabled	false	(Experimental) If set to "true", Spark will exclude the executor immediately when a fetch failure happens. If external shuffle service is enabled, then the whole node will be excluded.	2.3.0
spark.speculation	false	If set to "true", performs speculative execution of tasks. This means if one or more tasks are running slowly in a stage, they will be re-launched.	0.6.0
spark.speculation.interval	100ms	How often Spark will check for tasks to speculate.	0.6.0
spark.speculation.multiplier	3	How many times slower a task is than the median to be considered for speculation.	0.6.0
spark.speculation.quantile	0.9	Fraction of tasks which must be complete before speculation is enabled for a particular stage.	0.6.0
spark.speculation.minTaskRuntime	100ms	Minimum amount of time a task runs before being considered for speculation. This can be used to avoid launching speculative copies of tasks that are very short.	3.2.0
spark.speculation.task.duration.threshold	None	Task duration after which scheduler would try to speculative run the task. If provided, tasks would be speculatively run if current stage contains less tasks than or equal to the number of slots on a single executor and the task is taking longer time than the threshold. This config helps speculate stage with very few tasks. Regular speculation configs may also apply if the executor slots are large enough. E.g. tasks might be re-launched if there are enough successful runs even though the threshold hasn't been reached. The number of slots is computed based on the conf values of spark.executor.cores and spark.task.cpus minimum 1. Default unit is bytes, unless otherwise specified.	3.0.0
spark.speculation.efficiency.processRateMultiplier	0.75	A multiplier that used when evaluating inefficient tasks. The higher the multiplier is, the more tasks will be possibly considered as inefficient.	3.4.0
spark.speculation.efficiency.longRunTaskFactor	2	A task will be speculated anyway as long as its duration has exceeded the value of multiplying the factor and the time threshold (either be spark.speculation.multiplier * successfulTaskDurations.median or spark.speculation.minTaskRuntime) regardless of it's data process rate is good or not. This avoids missing the inefficient tasks when task slow isn't related to data process rate.	3.4.0
spark.speculation.efficiency.enabled	true	When set to true, spark will evaluate the efficiency of task processing through the stage task metrics or its duration, and only need to speculate the inefficient tasks. A task is inefficient when 1)its data process rate is less than the average data process rate of all successful tasks in the stage multiplied by a multiplier or 2)its duration has exceeded the value of multiplying spark.speculation.efficiency.longRunTaskFactor and the time threshold (either be spark.speculation.multiplier * successfulTaskDurations.median or spark.speculation.minTaskRuntime).	3.4.0
spark.task.cpus	1	Number of cores to allocate for each task.	0.5.0
spark.task.resource.{resourceName}.amount	1	Amount of a particular resource type to allocate for each task, note that this can be a double. If this is specified you must also provide the executor config spark.executor.resource.{resourceName}.amount and any corresponding discovery configs so that your executors are created with that resource type. In addition to whole amounts, a fractional amount (for example, 0.25, which means 1/4th of a resource) may be specified. Fractional amounts must be less than or equal to 0.5, or in other words, the minimum amount of resource sharing is 2 tasks per resource. Additionally, fractional amounts are floored in order to assign resource slots (e.g. a 0.2222 configuration, or 1/0.2222 slots will become 4 tasks/resource, not 5).	3.0.0
spark.task.maxFailures	4	Number of continuous failures of any particular task before giving up on the job. The total number of failures spread across different tasks will not cause the job to fail; a particular task has to fail this number of attempts continuously. If any attempt succeeds, the failure count for the task will be reset. Should be greater than or equal to 1. Number of allowed retries = this value - 1.	0.8.0
spark.task.reaper.enabled	false	Enables monitoring of killed / interrupted tasks. When set to true, any task which is killed will be monitored by the executor until that task actually finishes executing. See the other spark.task.reaper.* configurations for details on how to control the exact behavior of this monitoring. When set to false (the default), task killing will use an older code path which lacks such monitoring.	2.0.3
spark.task.reaper.pollingInterval	10s	When spark.task.reaper.enabled = true , this setting controls the frequency at which executors will poll the status of killed tasks. If a killed task is still running when polled then a warning will be logged and, by default, a thread-dump of the task will be logged (this thread dump can be disabled via the spark.task.reaper.threadDump setting, which is documented below).	2.0.3
spark.task.reaper.threadDump	true	When spark.task.reaper.enabled = true , this setting controls whether task thread dumps are logged during periodic polling of killed tasks. Set this to false to disable collection of thread dumps.	2.0.3
spark.task.reaper.killTimeout	-1	When spark.task.reaper.enabled = true , this setting specifies a timeout after which the executor JVM will kill itself if a killed task has not stopped running. The default value, -1, disables this mechanism and prevents the executor from self-destructing. The purpose of this setting is to act as a safety-net to prevent runaway noncancellable tasks from rendering an executor unusable.	2.0.3
spark.stage.maxConsecutiveAttempts	4	Number of consecutive stage attempts allowed before a stage is aborted.	2.2.0
spark.stage.ignoreDecommissionFetchFailure	true	Whether ignore stage fetch failure caused by executor decommission when count spark.stage.maxConsecutiveAttempts	3.4.0
spark.barrier.sync.timeout	365d	The timeout in seconds for each barrier() call from a barrier task. If the coordinator didn't receive all the sync messages from barrier tasks within the configured time, throw a SparkException to fail all the tasks. The default value is set to 31536000(3600 * 24 * 365) so the barrier() call shall wait for one year.	2.4.0

Configuration	Default	Description	Since
spark.scheduler.barrier.maxConcurrentTasksCheck.interval	15s	Time in seconds to wait between a max concurrent tasks check failure and the next check. A max concurrent tasks check ensures the cluster can launch more concurrent tasks than required by a barrier stage on job submitted. The check can fail in case a cluster has just started and not enough executors have registered, so we wait for a little while and try to perform the check again. If the check fails more than a configured max failure times for a job then fail current job submission. Note this config only applies to jobs that contain one or more barrier stages, we won't perform the check on non-barrier jobs.	2.4.0
spark.scheduler.barrier.maxConcurrentTasksCheck.maxFailures	40	Number of max concurrent tasks check failures allowed before fail a job submission. A max concurrent tasks check ensures the cluster can launch more concurrent tasks than required by a barrier stage on job submitted. The check can fail in case a cluster has just started and not enough executors have registered, so we wait for a little while and try to perform the check again. If the check fails more than a configured max failure times for a job then fail current job submission. Note this config only applies to jobs that contain one or more barrier stages, we won't perform the check on non-barrier jobs.	2.4.0
spark.dynamicAllocation.enabled	false	Whether to use dynamic resource allocation, which scales the number of executors registered with this application up and down based on the workload. For more detail, see the description here . This requires one of the following conditions: 1) enabling external shuffle service through spark.shuffle.service.enabled , or 2) enabling shuffle tracking through spark.dynamicAllocation.shuffleTracking.enabled , or 3) enabling shuffle blocks decommission through spark.decommission.enabled and spark.storage.decommission.shuffleBlocks.enabled , or 4) (Experimental) configuring spark.shuffle.sort.io.plugin.class to use a custom ShuffleDataIO who's ShuffleDriverComponents supports reliable storage. The following configurations are also relevant: spark.dynamicAllocation.minExecutors , spark.dynamicAllocation.maxExecutors , and spark.dynamicAllocation.initialExecutors spark.dynamicAllocation.executorAllocationRatio	1.2.0
spark.dynamicAllocation.executorIdleTimeout	60s	If dynamic allocation is enabled and an executor has been idle for more than this duration, the executor will be removed. For more detail, see this description .	1.2.0
spark.dynamicAllocation.cachedExecutorIdleTimeout	infinity	If dynamic allocation is enabled and an executor which has cached data blocks has been idle for more than this duration, the executor will be removed. For more details, see this description .	1.4.0
spark.dynamicAllocation.initialExecutors	spark.dynamicAllocation.minExecutors	Initial number of executors to run if dynamic allocation is enabled. If --num-executors (or spark.executor.instances) is set and larger than this value, it will be used as the initial number of executors.	1.3.0
spark.dynamicAllocation.maxExecutors	infinity	Upper bound for the number of executors if dynamic allocation is enabled.	1.2.0
spark.dynamicAllocation.minExecutors	0	Lower bound for the number of executors if dynamic allocation is enabled.	1.2.0
spark.dynamicAllocation.executorAllocationRatio	1	By default, the dynamic allocation will request enough executors to maximize the parallelism according to the number of tasks to process. While this minimizes the latency of the job, with small tasks this setting can waste a lot of resources due to executor allocation overhead, as some executor might not even do any work. This setting allows to set a ratio that will be used to reduce the number of executors w.r.t. full parallelism. Defaults to 1.0 to give maximum parallelism. 0.5 will divide the target number of executors by 2 The target number of executors computed by the dynamicAllocation can still be overridden by the spark.dynamicAllocation.minExecutors and spark.dynamicAllocation.maxExecutors settings	2.4.0
spark.dynamicAllocation.schedulerBacklogTimeout	1s	If dynamic allocation is enabled and there have been pending tasks backlogged for more than this duration, new executors will be requested. For more detail, see this description .	1.2.0
spark.dynamicAllocation.sustainedSchedulerBacklogTimeout	schedulerBacklogTimeout	Same as spark.dynamicAllocation.schedulerBacklogTimeout , but used only for subsequent executor requests. For more detail, see this description .	1.2.0
spark.dynamicAllocation.shuffleTracking.enabled	true	Enables shuffle file tracking for executors, which allows dynamic allocation without the need for an external shuffle service. This option will try to keep alive executors that are storing shuffle data for active jobs.	3.0.0
spark.dynamicAllocation.shuffleTracking.timeout	infinity	When shuffle tracking is enabled, controls the timeout for executors that are holding shuffle data. The default value means that Spark will rely on the shuffles being garbage collected to be able to release executors. If for some reason garbage collection is not cleaning up shuffles quickly enough, this option can be used to control when to time out executors even when they are storing shuffle data.	3.0.0
spark.{driver|executor}.rpc.io.serverThreads	Fall back on spark.rpc.io.serverThreads	Number of threads used in the server thread pool	1.6.0
spark.{driver|executor}.rpc.io.clientThreads	Fall back on spark.rpc.io.clientThreads	Number of threads used in the client thread pool	1.6.0
spark.{driver|executor}.rpc.netty.dispatcher.numThreads	Fall back on spark.rpc.netty.dispatcher.numThreads	Number of threads used in RPC message dispatcher thread pool	3.0.0
spark.api.mode	classic	For Spark Classic applications, specify whether to automatically use Spark Connect by running a local Spark Connect server. The value can be classic or connect .	4.0.0
spark.connect.grpc.binding.address	(none)	Address for Spark Connect server to bind.	4.0.0
spark.connect.grpc.binding.port	15002	Port for Spark Connect server to bind.	3.4.0
spark.connect.grpc.port.maxRetries	0	The max port retry attempts for the gRPC server binding. By default, it's set to 0, and the server will fail fast in case of port conflicts.	4.0.0
spark.connect.grpc.interceptor.classes	(none)	Comma separated list of class names that must implement the io.grpc.ServerInterceptor interface	3.4.0
spark.connect.grpc.arrow.maxBatchSize	4m	When using Apache Arrow, limit the maximum size of one arrow batch that can be sent from server side to client side. Currently, we conservatively use 70% of it because the size is not accurate but estimated.	3.4.0
spark.connect.grpc.maxInboundMessageSize	134217728	Sets the maximum inbound message size for the gRPC requests. Requests with a larger payload will fail.	3.4.0
spark.connect.extensions.relation.classes	(none)	Comma separated list of classes that implement the trait org.apache.spark.sql.connect.plugin.RelationPlugin to support custom Relation types in proto.	3.4.0
spark.connect.extensions.expression.classes	(none)	Comma separated list of classes that implement the trait org.apache.spark.sql.connect.plugin.ExpressionPlugin to support custom Expression types in proto.	3.4.0
spark.connect.extensions.command.classes	(none)	Comma separated list of classes that implement the trait org.apache.spark.sql.connect.plugin.CommandPlugin to support custom Command types in proto.	3.4.0
spark.connect.ml.backend.classes	(none)	Comma separated list of classes that implement the trait org.apache.spark.sql.connect.plugin.MLBackendPlugin to replace the specified Spark ML operators with a backend-specific implementation.	4.0.0

Configuration	Default	Description	Since
spark.connect.jvmStacktrace.maxSize	1024	Sets the maximum stack trace size to display when `spark.sql.pyspark.jvmStacktrace.enabled` is true.	3.5.0
spark.sql.connect.ui.retainedSessions	200	The number of client sessions kept in the Spark Connect UI history.	3.5.0
spark.sql.connect.ui.retainedStatements	200	The number of statements kept in the Spark Connect UI history.	3.5.0
spark.sql.connect.enrichError.enabled	true	When true, it enriches errors with full exception messages and optionally server-side stacktrace on the client side via an additional RPC.	4.0.0
spark.sql.connect.serverStacktrace.enabled	true	When true, it sets the server-side stacktrace in the user-facing Spark exception.	4.0.0
spark.connect.grpc.maxMetadataSize	1024	Sets the maximum size of metadata fields. For instance, it restricts metadata fields in `ErrorInfo`.	4.0.0
spark.connect.progress.reportInterval	2s	The interval at which the progress of a query is reported to the client. If the value is set to a negative value the progress reports will be disabled.	4.0.0
spark.streaming.backpressure.enabled	false	Enables or disables Spark Streaming's internal backpressure mechanism (since 1.5). This enables the Spark Streaming to control the receiving rate based on the current batch scheduling delays and processing times so that the system receives only as fast as the system can process. Internally, this dynamically sets the maximum receiving rate of receivers. This rate is upper bounded by the values spark.streaming.receiver.maxRate and spark.streaming.kafka.maxRatePerPartition if they are set (see below).	1.5.0
spark.streaming.backpressure.initialRate	not set	This is the initial maximum receiving rate at which each receiver will receive data for the first batch when the backpressure mechanism is enabled.	2.0.0
spark.streaming.blockInterval	200ms	Interval at which data received by Spark Streaming receivers is chunked into blocks of data before storing them in Spark. Minimum recommended - 50 ms. See the performance tuning section in the Spark Streaming programming guide for more details.	0.8.0
spark.streaming.receiver.maxRate	not set	Maximum rate (number of records per second) at which each receiver will receive data. Effectively, each stream will consume at most this number of records per second. Setting this configuration to 0 or a negative number will put no limit on the rate. See the deployment guide in the Spark Streaming programming guide for mode details.	1.0.2
spark.streaming.receiver.writeAheadLog.enable	false	Enable write-ahead logs for receivers. All the input data received through receivers will be saved to write-ahead logs that will allow it to be recovered after driver failures. See the deployment guide in the Spark Streaming programming guide for more details.	1.2.1
spark.streaming.unpersist	true	Force RDDs generated and persisted by Spark Streaming to be automatically unpersisted from Spark's memory. The raw input data received by Spark Streaming is also automatically cleared. Setting this to false will allow the raw data and persisted RDDs to be accessible outside the streaming application as they will not be cleared automatically. But it comes at the cost of higher memory usage in Spark.	0.9.0
spark.streaming.stopGracefullyOnShutdown	false	If true , Spark shuts down the StreamingContext gracefully on JVM shutdown rather than immediately.	1.4.0
spark.streaming.kafka.maxRatePerPartition	not set	Maximum rate (number of records per second) at which data will be read from each Kafka partition when using the new Kafka direct stream API. See the Kafka Integration guide for more details.	1.3.0
spark.streaming.kafka.minRatePerPartition	1	Minimum rate (number of records per second) at which data will be read from each Kafka partition when using the new Kafka direct stream API.	2.4.0
spark.streaming.ui.retainedBatches	1000	How many batches the Spark Streaming UI and status APIs remember before garbage collecting.	1.0.0
spark.streaming.driver.writeAheadLog.closeFileAfterWrite	false	Whether to close the file after writing a write-ahead log record on the driver. Set this to 'true' when you want to use S3 (or any file system that does not support flushing) for the metadata WAL on the driver.	1.6.0
spark.streaming.receiver.writeAheadLog.closeFileAfterWrite	false	Whether to close the file after writing a write-ahead log record on the receivers. Set this to 'true' when you want to use S3 (or any file system that does not support flushing) for the data WAL on the receivers.	1.6.0
spark.r.numRBackendThreads	2	Number of threads used by RBackend to handle RPC calls from SparkR package.	1.4.0
spark.r.command	Rscript	Executable for executing R scripts in cluster modes for both driver and workers.	1.5.3
spark.r.driver.command	spark.r.command	Executable for executing R scripts in client modes for driver. Ignored in cluster modes.	1.5.3
spark.r.shell.command	R	Executable for executing sparkR shell in client modes for driver. Ignored in cluster modes. It is the same as environment variable SPARKR_DRIVER_R , but take precedence over it. spark.r.shell.command is used for sparkR shell while spark.r.driver.command is used for running R script.	2.1.0
spark.r.backendConnectionTimeout	6000	Connection timeout set by R process on its connection to RBackend in seconds.	2.1.0
spark.r.heartBeatInterval	100	Interval for heartbeats sent from SparkR backend to R process to prevent connection timeout.	2.1.0
spark.graphx.pregel.checkpointInterval	-1	Checkpoint interval for graph and message in Pregel. It used to avoid stackOverflowError due to long lineage chains after lots of iterations. The checkpoint is disabled by default.	2.2.0
spark.shuffle.push.server.mergedShuffleFileManagerImpl	org.apache.spark.network.shuffle.NoOpMergedShuffleFileManager	Class name of the implementation of MergedShuffleFileManager that manages push-based shuffle. This acts as a server side config to disable or enable push-based shuffle. By default, push-based shuffle is disabled at the server side. To enable push-based shuffle on the server side, set this config to org.apache.spark.network.shuffle.RemoteBlockPushResolver	3.2.0
spark.shuffle.push.server.minChunkSizeInMergedShuffleFile	2m	The minimum size of a chunk when dividing a merged shuffle file into multiple chunks during push-based shuffle. A merged shuffle file consists of multiple small shuffle blocks. Fetching the complete merged shuffle file in a single disk I/O increases the memory requirements for both the clients and the external shuffle services. Instead, the external shuffle service serves the merged file in MB-sized chunks . This configuration controls how big a chunk can get. A corresponding index file for each merged shuffle file will be generated indicating chunk boundaries. Setting this too high would increase the memory requirements on both the clients and the external shuffle service. Setting this too low would increase the overall number of RPC requests to external shuffle service unnecessarily.	3.2.0
spark.shuffle.push.server.mergedIndexCacheSize	100m	The maximum size of cache in memory which could be used in push-based shuffle for storing merged index files. This cache is in addition to the one configured via spark.shuffle.service.index.cache.size .	3.2.0
spark.shuffle.push.enabled	false	Set to true to enable push-based shuffle on the client side and works in conjunction with the server side flag spark.shuffle.push.server.mergedShuffleFileManagerImpl .	3.2.0

Configuration	Default	Description	Since
spark.shuffle.push.finalize.timeout	10s	The amount of time driver waits in seconds, after all mappers have finished for a given shuffle map stage, before it sends merge finalize requests to remote external shuffle services. This gives the external shuffle services extra time to merge blocks. Setting this too long could potentially lead to performance regression.	3.2.0
spark.shuffle.push.maxRetainedMergerLocations	500	Maximum number of merger locations cached for push-based shuffle. Currently, merger locations are hosts of external shuffle services responsible for handling pushed blocks, merging them and serving merged blocks for later shuffle fetch.	3.2.0
spark.shuffle.push.mergersMinThresholdRatio	0.05	Ratio used to compute the minimum number of shuffle merger locations required for a stage based on the number of partitions for the reducer stage. For example, a reduce stage which has 100 partitions and uses the default value 0.05 requires at least 5 unique merger locations to enable push-based shuffle.	3.2.0
spark.shuffle.push.mergersMinStaticThreshold	5	The static threshold for number of shuffle push merger locations should be available in order to enable push-based shuffle for a stage. Note this config works in conjunction with spark.shuffle.push.mergersMinThresholdRatio . Maximum of spark.shuffle.push.mergersMinStaticThreshold and spark.shuffle.push.mergersMinThresholdRatio ratio number of mergers needed to enable push-based shuffle for a stage. For example: with 1000 partitions for the child stage with spark.shuffle.push.mergersMinStaticThreshold as 5 and spark.shuffle.push.mergersMinThresholdRatio set to 0.05, we would need at least 50 mergers to enable push-based shuffle for that stage.	3.2.0
spark.shuffle.push.numPushThreads	(none)	Specify the number of threads in the block pusher pool. These threads assist in creating connections and pushing blocks to remote external shuffle services. By default, the threadpool size is equal to the number of spark executor cores.	3.2.0
spark.shuffle.push.maxBlockSizeToPush	1m	The max size of an individual block to push to the remote external shuffle services. Blocks larger than this threshold are not pushed to be merged remotely. These shuffle blocks will be fetched in the original manner. Setting this too high would result in more blocks to be pushed to remote external shuffle services but those are already efficiently fetched with the existing mechanisms resulting in additional overhead of pushing the large blocks to remote external shuffle services. It is recommended to set spark.shuffle.push.maxBlockSizeToPush lesser than spark.shuffle.push.maxBlockBatchSize config's value. Setting this too low would result in lesser number of blocks getting merged and directly fetched from mapper external shuffle service results in higher small random reads affecting overall disk I/O performance.	3.2.0
spark.shuffle.push.maxBlockBatchSize	3m	The max size of a batch of shuffle blocks to be grouped into a single push request. Default is set to 3m in order to keep it slightly higher than spark.storage.memoryMapThreshold default which is 2m as it is very likely that each batch of block gets memory mapped which incurs higher overhead.	3.2.0
spark.shuffle.push.merge.finalizeThreads	8	Number of threads used by driver to finalize shuffle merge. Since it could potentially take seconds for a large shuffle to finalize, having multiple threads helps driver to handle concurrent shuffle merge finalize requests when push-based shuffle is enabled.	3.3.0
spark.shuffle.push.minShuffleSizeToWait	500m	Driver will wait for merge finalization to complete only if total shuffle data size is more than this threshold. If total shuffle size is less, driver will immediately finalize the shuffle output.	3.3.0
spark.shuffle.push.minCompletedPushRatio	1.0	Fraction of minimum map partitions that should be push complete before driver starts shuffle merge finalization during push based shuffle.	3.3.0

Important notes

• Spark configuration is version-specific. This PDF is for Apache Spark 4.0.1.

• Some defaults are conditional on deployment mode, cluster manager, operating system, or another configuration.

• Spark also has environment variables and logging configuration that are not Spark properties; they are documented separately in the official configuration guide.

• The official documentation notes that explicitly supplied properties are shown in the Spark UI Environment tab; unspecified properties use their documented defaults.

Official source: Apache Spark 4.0.1 Configuration Documentation — https://spark.apache.org/docs/4.0.1/configuration.html